📐 Arquitetura — Investment Notifier App

> Java 21 + Spring Boot 3.x | Engenharia de Software Sênior

---

1. Visão Geral

Aplicação monolítica modular que monitora notícias de ações salvas pelo usuário e dispara notificações em redes sociais (ex: Telegram). Utiliza Spring Security para autenticação/autorização, scheduler para polling automático e client HTTP para consulta de APIs externas.

```
[ Usuário ] ──▶ [ API REST / Spring Security ]
                          │
            ┌─────────────┼──────────────┐
            ▼             ▼              ▼
     [Stock CRUD]  [News Fetcher]  [Notifier]
            │             │              │
            └─────────────┼──────────────┘
                          ▼
                   [ PostgreSQL ]
```

---

2. Stack Tecnológica

Camada	Tecnologia
Linguagem	Java 21 (Records, Virtual Threads)
Framework	Spring Boot 3.x
Segurança	Spring Security + JWT
Scheduler	Spring `@Scheduled`
HTTP Client	Spring WebClient (WebFlux)
Persistência	Spring Data JPA + PostgreSQL
Notificações	Telegram Bot API
Build	Maven
Testes	JUnit 5 + Mockito + Testcontainers

3. Estrutura de Pacotes

```
com.investmentnotifier
├── config/
|   ├── SecurityConfig.java           # Spring Security + JWT filter
|   ├── WebClientConfig.java          # Beans de WebClient
|   └── SchedulerConfig.java          # Habilita @Scheduled
|
├── domain/
|   ├── stock/
|   |   ├── Stock.java                # Entidade JPA
|   |   ├── StockRepository.java
|   |   ├── StockService.java
|   |   └── StockController.java
|   |
|   ├── news/
|   |   ├── News.java                 # Entidade / VO
|   |   ├── NewsRepository.java
|   |   ├── NewsService.java
|   |   └── NewsFetcherJob.java       # @Scheduled
|   |
|   └── notification/
|       ├── NotificationService.java  # Interface
|       ├── TelegramNotifier.java     # Implementação
|       └── NotificationFactory.java  # Factory
|
├── infra/
|   ├── external/
|   |   ├── StockApiClient.java       # WebClient → API externa
|   |   └── StockApiResponse.java     # DTO de resposta
|   └── security/
|       ├── JwtTokenProvider.java
|       ├── JwtAuthFilter.java
|       └── UserDetailsServiceImpl.java
|
└── shared/
    ├── exception/GlobalExceptionHandler.java
    └── dto/ApiResponse.java
```

4. Design Patterns Aplicados

4.1 Creational Patterns (Criação de objetos)

✅ **Factory Method — `NotificationFactory`**

Onde: `notification/NotificationFactory.java`

Por quê: A aplicação pode evoluir para suportar múltiplos canais (Telegram, Twitter/X, WhatsApp). A factory encapsula a lógica de instanciação do notificador correto com base em configuração.

```java
public interface NotificationService {
    void notify(String message, String recipient);
}

@Component
public class TelegramNotifier implements NotificationService {
    @Override
    public void notify(String message, String recipient) {
        // chama Telegram Bot API
    }
}

@Component
public class NotificationFactory {
    private final Map<String, NotificationService> notifiers;

    public NotificationFactory(List<NotificationService> services) {
        // Spring injeta todas as implementações automaticamente
        this.notifiers = services.stream()
            .collect(Collectors.toMap(
                s -> s.getClass().getSimpleName(),
                s -> s
            ));
    }

    public NotificationService get(String channel) {
        return Optional.ofNullable(notifiers.get(channel))
            .orElseThrow(() -> new IllegalArgumentException("Canal inválido: " + cha
    }
}
```

✅ **Builder — `News` / `ApiResponse`**

Onde: DTOs e Value Objects da aplicação

Por quê: Objetos com muitos campos opcionais ficam mais legíveis e seguros com Builder. Em Java 21, Records eliminam boilerplate mas para objetos mutáveis/complexos o Builder ainda é preferível.

```java
// Com Record (imutável) — preferível para DTOs
public record StockApiResponse(
    String ticker,
    String name,
    BigDecimal price,
    String exchange
) {}

// Com Builder — para entidades mais complexas
@Builder
@Entity
public class Stock {
    @Id @GeneratedValue
    private Long id;
    private String ticker;
    private String name;
    private boolean active;
    // ...
}
```

4.2 Structural Patterns (Estrutura e composição)

✅ **Adapter — `StockApiClient`**

Onde: `infra/external/StockApiClient.java`

Por quê: A API externa retorna um formato próprio (`StockApiResponse`). O Adapter traduz esse contrato externo para o modelo de domínio interno (`Stock`), isolando o restante da aplicação de mudanças na API de terceiros.

```java
```

```java
@Component
@RequiredArgsConstructor
public class StockApiClient {

    private final WebClient webClient;

    public List<Stock> searchStocks(String query) {
        List<StockApiResponse> responses = webClient.get()
            .uri("/stocks?query={q}", query)
            .retrieve()
            .bodyToFlux(StockApiResponse.class)
            .collectList()
            .block();

        return adapt(responses); // ← Adapter: converte DTO externo → domínio
    }

    private List<Stock> adapt(List<StockApiResponse> responses) {
        return responses.stream()
            .map(r -> Stock.builder()
                .ticker(r.ticker())
                .name(r.name())
                .build())
            .toList();
    }
}
```

✅ Facade — `StockService`

Onde: `domain/stock/StockService.java`

Por quê: O controller não precisa conhecer o `StockApiClient`, o `StockRepository` e as regras de negócio separadamente. O `StockService` age como Facade, expondo uma API simples para o controller.

```java
```

```java
@Service
@RequiredArgsConstructor
public class StockService {

    private final StockRepository repository;
    private final StockApiClient apiClient;      // infra externa

    // Facade: controller chama apenas este método
    public List<StockDTO> search(String query) {
        return apiClient.searchStocks(query)
            .stream()
            .map(StockDTO::from)
            .toList();
    }

    public Stock save(Stock stock) {
        return repository.save(stock);
    }

    public List<Stock> findAll() {
        return repository.findAll();
    }

    public void delete(Long id) {
        repository.deleteById(id);
    }
}
```

4.3 Behavioral Patterns (Comportamento e comunicação)

✅ **Observer / Event-Driven — `ApplicationEventPublisher`**

Onde: `news/NewsService.java` → `notification/NotificationService.java`

Por quê: Quando uma nova notícia é detectada, o `NewsService` não deve acoplar diretamente ao `TelegramNotifier`. Usando o `ApplicationEventPublisher` do Spring (Observer nativo), a publicação e o consumo ficam desacoplados — fácil de adicionar novos listeners sem tocar no publicador.

```java
```

```java
// Evento de domínio
public record NewsDetectedEvent(Stock stock, String headline, String url) {}

// Publicador (Observer Subject)
@Service
@RequiredArgsConstructor
public class NewsService {

    private final ApplicationEventPublisher eventPublisher;

    public void processNews(Stock stock, String headline, String url) {
        // persiste a notícia...
        eventPublisher.publishEvent(new NewsDetectedEvent(stock, headline, url));
    }
}

// Consumidor (Observer)
@Component
@RequiredArgsConstructor
public class NotificationEventListener {

    private final NotificationFactory factory;

    @EventListener
    @Async
    public void onNewsDetected(NewsDetectedEvent event) {
        String msg = "📈 Nova notícia sobre %s: %s\n%s"
            .formatted(event.stock().getTicker(), event.headline(), event.url());

        factory.get("TelegramNotifier")
               .notify(msg, "@canal_investimentos");
    }
}
```

✅ Template Method — `NewsScheduler`

Onde: `news/NewsFetcherJob.java`

Por quê: O fluxo de polling é sempre: *buscar ações salvas → consultar API de notícias → filtrar novas → publicar evento*. O Template Method define esse esqueleto fixo e permite que subclasses ou lambdas customizem etapas específicas (ex: fonte da notícia).

```java
```

```java
@Component
@RequiredArgsConstructor
public class NewsSchedulerJob {

    private final StockRepository stockRepository;
    private final NewsService newsService;

    @Scheduled(fixedDelayString = "${app.scheduler.interval-ms:300000}")
    public void run() {
        List<Stock> activeStocks = stockRepository.findByActiveTrue(); // passo 1
        activeStocks.forEach(this::checkNews);                         // passo 2
    }

    // Template: esqueleto fixo
    private void checkNews(Stock stock) {
        fetchHeadlines(stock)              // ← customizável por fonte
            .stream()
            .filter(this::isNew)
            .forEach(h -> newsService.processNews(stock, h.title(), h.url()));
    }

    // Pode ser sobrescrito/injetado para diferentes fontes
    protected List<Headline> fetchHeadlines(Stock stock) {
        // chama API externa de notícias
    }

    private boolean isNew(Headline h) {
        // verifica se já foi processada
    }
}
```

---

5. Spring Security — Fluxo JWT (Didático)

```
POST /auth/login
      |
      ▼
JwtAuthFilter (OncePerRequestFilter)
      |  extrai Bearer Token do header
      ▼
JwtTokenProvider.validateToken()
      |  válido?
      ├── Não → 401 Unauthorized
      |
      └── Sim → seta SecurityContextHolder
```

```
                        |
                        ▼
                Controller executa
```

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.cla
            .build();
    }
}
```

---

6. Fluxo Completo de uma Notificação

```
[Scheduler @5min]
      |
      ▼
NewsFetcherJob.run()
      |  busca stocks ativos
      ▼
StockRepository.findByActiveTrue()
      |
      ▼ para cada stock
NewsApiClient.fetchHeadlines(stock)
      |
      ▼
NewsService.processNews()  ──► [ persiste News ]
      |
      ▼
ApplicationEventPublisher.publishEvent(NewsDetectedEvent)
      |
      ▼
```

```
NotificationEventListener.onNewsDetected()  [@Async]
        |
        ▼
NotificationFactory.get("TelegramNotifier")
        |
        ▼
TelegramNotifier.notify()  ──▶ 📲 Telegram
```

---

7. Resumo dos Padrões

Categoria	Pattern	Classe Principal	Justificativa
Creational	Factory Method	`NotificationFactory`	Suporte a múltiplos canais de notificação
Creational	Builder	`Stock`, `ApiResponse`	Construção segura de objetos com muitos campos
Structural	Adapter	`StockApiClient`	Isola contrato da API externa do domínio
Structural	Facade	`StockService`	Simplifica interface para o controller
Behavioral	Observer	`ApplicationEventPublisher` + `EventListener`	Desacoplamento entre detecção de notícia e envio
Behavioral	Template Method	`NewsSchedulerJob`	Esqueleto fixo de polling com passos customizáveis

---

8. Decisões de Design (ADRs Simplificados)

ADR-01 — Monólito Modular vs. Microsserviços Escolhido monólito modular. O domínio é pequeno e os trade-offs de complexidade operacional de microsserviços não se justificam neste estágio.

ADR-02 — WebClient vs. RestTemplate Escolhido `WebClient` mesmo em contexto síncrono, pois é o padrão atual do Spring (RestTemplate está em modo manutenção) e permite migração para reativo no futuro.

ADR-03 — Eventos internos vs. Message Broker Escolhido `ApplicationEventPublisher` (síncrono/assíncrono via `@Async`). Um broker como Kafka/RabbitMQ seria over-engineering para o volume atual — pode ser adicionado se a escala exigir.

ADR-04 — JWT stateless Alinha com a natureza REST da API e é didaticamente valioso para aprender Spring Security sem dependência de sessão HTTP.